

NEUROPLANT

Division of Labour

ET AI Hackathon 2026 | PS8 — AI for Industrial Knowledge Intelligence | Team Reference
Document | Python/FastAPI + Next.js Stack

Overview — Who Owns What

Member	Role	Files / Modules You Own	Files You Never Touch
P1	Backend Lead (Application Layer, Auth, Org/Plant/Document/Notification/Admin/Audit Services, DevOps)	backend/api backend/auth backend/core backend/database backend/models backend/schemas backend/services (org, plant, user, document, notification, admin, audit, report) backend/workers docker/, scripts/, docker-compose.yml	backend/agents, backend/graph, backend/ai, backend/ingestion, backend/compliance, backend/decision, frontend/, shared/prompts
P2	AI & Knowledge Engineering Lead (Intelligence Layer, GraphRAG, Knowledge Graph, Integrity Engines)	backend/agents backend/graph backend/ai backend/ingestion backend/compliance (AI logic) backend/decision shared/prompts	backend/api, backend/auth, frontend/, docker/, docs/ (final ownership)
P3	Frontend Lead (Experience Layer, Dashboards, Onboarding, Graph Explorer, Decision Assistant UI)	frontend/app frontend/components frontend/features frontend/hooks frontend/lib frontend/services frontend/types frontend/utils frontend/public	Any backend/ folder, shared/prompts, Neo4j / Postgres schema files
P4	Architecture & Product Lead (Chief Architect — Diagrams, Docs, Demo Dataset, Presentation, PR Review, Judge Prep)	docs/architecture docs/api docs/diagrams	Does not write feature code directly — reviews and

		docs/datasets (sample/demo dataset) Presentation deck, demo script, ER & Neo4j schema docs (design authority, not implementation)	protects architectural consistency across all of the above
--	--	---	--

Golden Rule

You own your module. You never silently redesign someone else's. The only shared, immutable things are: (1) the Shared Contracts section in this document (Postgres schema, Neo4j labels/relationships, API envelope, locked endpoints), (2) docs/datasets/ sample data which Person 4 owns but everyone consumes, and (3) the Architecture Rules in the Core Philosophy section, which apply to all four members at all times.

Core Philosophy — Non-Negotiable for All Four Members

Hackathon: ET AI Hackathon 2026 — Problem Statement 8 (AI for Industrial Knowledge Intelligence). Project: NeuroPlant. Timeline: 10 days. Team: 4 developers, AI-assisted.

We are NOT building

- A chatbot
- A PDF Question Answering system
- A document repository
- A simple RAG application
- A replacement for ERP / CMMS / EDMS / SharePoint / SAP

We ARE building

An Industrial Knowledge Intelligence Platform that continuously converts fragmented industrial knowledge into explainable operational intelligence using AI, Knowledge Graphs, GraphRAG, and deterministic reasoning. Documents are evidence. Knowledge is the product. Operational Intelligence is the output.

Product Values

- Trust over confidence
- Evidence over assumptions
- Knowledge over documents
- Explainability over black-box AI
- Operational safety over automation
- Knowledge preservation over knowledge accumulation
- Enterprise quality over hackathon shortcuts

Architecture Rules (apply to every member, every PR)

- AI extracts information. Graphs organize information. Rules verify information. Humans approve critical decisions.
- Documents remain evidence; Knowledge remains the source of truth.
- Neo4j is mandatory. GraphRAG is mandatory. Deterministic verification is mandatory.
- Multi-tenancy is mandatory — every table/node/request is scoped by organization_id.
- Decision Briefs are mandatory — the Decision Assistant never returns a plain paragraph.
- Knowledge Integrity Engine, Knowledge Mortality Score, and Regulatory Drift Detection are mandatory.
- Never allow an LLM to determine regulatory compliance by itself — compliance is always deterministic-rule-verified.
- No relationship is ever invented in the Knowledge Graph without a supporting evidence/source reference.

What the AI / Any Member Should Never Do

- Redesign the architecture without a fundamental flaw being demonstrated

- Convert the platform into a chatbot
- Remove Neo4j or GraphRAG
- Replace deterministic verification with LLM reasoning
- Introduce unnecessary technologies or over-engineer beyond hackathon scope
- Fabricate compliance decisions or invent graph relationships without evidence
- Break multi-tenancy or ignore role-based access control

What Every Member Should Always Do

- Preserve architectural consistency; think like a Staff Engineer / Enterprise Architect
- Think about scalability, maintainability, and explainability
- Prefer enterprise-quality solutions that remain realistic within the 10-day window
- Improve implementation without changing product vision; raise concerns to Person 4 before any major change

Universal Coding Standards

- REST APIs everywhere. snake_case in Python. camelCase in TypeScript. PascalCase for React components.
- Never place AI prompts inside controllers — store prompts inside dedicated prompt modules (shared/prompts).
- Business logic must never live inside API routes. Every service has a single responsibility. Use dependency injection.
- Never hardcode API keys or secrets — use environment variables (.env).
- Always validate AI output before storing it. Never trust LLM output blindly.
- Every API response uses the shared response envelope (see Shared Contracts).

Git Workflow

main → develop → feature/* — No direct commits to main. Suggested branch names: feature/auth, feature/document-upload, feature/neo4j, feature/graphrag, feature/frontend-dashboard, feature/compliance, feature/decision-brief. Every merge must preserve architectural consistency; Person 4 reviews PRs for architectural alignment.

Project Folder Structure

Each member works within their designated modules only — if your name is not next to a module above, you do not touch it without a cross-team conversation.

neuroplant/	
frontend/	← P3 owns entirely
app/ components/ features/ hooks/ lib/ services/ types/ utils/ public/	
backend/	
api/	← P1 (route layer, request/response only)
auth/	← P1 (JWT, RBAC)
core/	← P1 (config, settings, DI containers)
database/	← P1 (PostgreSQL access layer)
models/ schemas/	← P1 (Pydantic / ORM models)
services/ Admin services)	← P1 (Org, Plant, User, Document, Notification, Report, Audit,
agents/ Expert Interview Agent)	← P2 (Document Classifier, Entity/Relationship Extraction agents,
graph/	← P2 (Neo4j Knowledge Graph Builder, graph population, traversal)
ai/ Engine)	← P2 (OCR Engine, Vision Parser, Embedding Generator, GraphRAG
ingestion/	← P2 (document ingestion pipeline orchestration)
compliance/ P1 exposes via API)	← P2 (Contradiction Detector, Regulatory Drift Engine — AI logic;
decision/	← P2 (Decision Brief Generator, Knowledge Mortality Engine)
notifications/	← P1 (alert dispatch)
workers/ they call)	← P1 (Celery / background task workers; P2 supplies the AI jobs
utils/	← P1
shared/	← jointly governed, P4 reviews any change
prompts/	← P2 owns content; never inline prompts in controllers
constants/ configs/ types/	← shared, change requires team agreement
docs/	← P4 owns entirely
architecture/ api/ diagrams/ datasets/	(sample/demo organization data)
docker/	← P1 owns; P4 reviews for deployment alignment

scripts/	← P1
tests/	← each member writes tests for their own modules
README.md docker-compose.yml	← P1 creates; changes require team agreement

Shared Contracts — Agree Day 1, Never Change

These contracts are immutable for all four members. If they are respected, integration on Day 6+ works without rediscovering mismatches.

PostgreSQL Core Tables (minimum)

organizations, plants, users, roles, documents, audit_logs.

- Every table includes organization_id as a foreign key for tenant isolation.
- Every table includes created_at and updated_at timestamps.

Neo4j Node Labels (locked list)

Equipment, Procedure, Regulation, Inspection, Incident, WorkOrder, MaintenanceHistory, Expert, ExpertInsight, Document, Plant, Organization, PIDElement.

Every node carries these mandatory properties: org_id, created_at, updated_at, confidence, source_document_id.

Neo4j Relationship Types (locked list)

CONNECTED_TO, PART_OF, LOCATED_IN, MAINTAINED_BY, GOVERNED_BY, REFERENCES, INSPECTED_BY, CAUSES, PREVENTS, DEPENDS_ON, DOWNSTREAM_AFFECTS, REQUIRES, AUTHORED_BY, CONTRADICTS, SUPERSEDES, RELATES_TO.

No relationship may be created outside this list, and none may be invented without supporting evidence.

API Response Envelope (every endpoint, no exceptions)

```
{
  "success": true,
  "data": {},
  "message": "string",
  "timestamp": "ISO8601"
}
```

Core API Endpoints — Locked Before Day 1

Owner: Person 1 builds the real implementation. Person 3 builds the entire frontend against these exact contracts using mock data from Day 1, so the two integrate cleanly without rediscovery.

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/organizations
- POST /api/v1/plants
- POST /api/v1/documents/upload
- GET /api/v1/documents/{id}/status

- GET /api/v1/dashboard/alerts
- GET /api/v1/integrity/contradictions
- GET /api/v1/integrity/regulatory-drift
- GET /api/v1/mortality/score
- POST /api/v1/decisions/query
- GET /api/v1/graph/nodes
- GET /api/v1/graph/relationships
- POST /api/v1/expert/interview/start

Dependency Flow

Member	Depends On	Provides To	Independent From Day 1 By
P1	Nothing — starts fresh	P2 (org_id-scoped DB access, JWT middleware, document storage hooks), P3 (locked REST contracts)	Seeding Postgres with a mock organization + plant + users locally; serving the locked endpoint contracts with mock responses
P2	P1's Postgres schema (organizations, plants, documents) and auth middleware	P1 (extracted entities/relationships to persist), P3 (Knowledge Graph nodes/edges, Decision Briefs, Integrity scores via API)	Running the ingestion pipeline and GraphRAG locally against the sample demo dataset (docs/datasets) without waiting on live Postgres
P3	P1's locked API contracts and response envelope; P2's Decision Brief / Knowledge Graph shapes	End user / judges (dashboards, onboarding, Graph Explorer, Decision Assistant UI)	Building 100% of the frontend against mocked JSON matching the locked envelope and endpoint list
P4	Visibility into all three other members' progress	The demo dataset (docs/datasets) all members build against, architecture diagrams, the presentation deck, and the demo script	Designing the sample organization (contradictions, compliance issues, incidents, expert knowledge) independently of any running code

Integration happens in phases starting mid-hackathon. Until then, all four members work independently against the Shared Contracts and the Person-4-authored demo dataset.

Member Responsibilities

P1

Backend Lead

Application Layer | Auth & RBAC |
Org/Plant/Document/Notification/Admin/Audit Services | DevOps

You are the foundation. Every other member depends on your Postgres schema, your JWT/RBAC middleware, and the locked API contracts. You own onboarding (register → login → create organization → create plant → upload), multi-tenant isolation, document storage, background job orchestration, notifications, audit logging, and the Docker/CI deployment pipeline. You never perform AI inference directly — you orchestrate and hand off to Person 2's Intelligence Layer.

Step	What to Build	Key Output / Module
1	PostgreSQL schema — organizations, plants, users, roles, documents, audit_logs (+ notifications, settings, API keys as needed). Every table gets organization_id, created_at, updated_at.	backend/database, backend/models
2	FastAPI app bootstrap — Uvicorn, Pydantic validation, OpenAPI docs, FastAPI Depends for DI, global error handler, CORS, request logging.	backend/core, backend/api
3	JWT auth + RBAC — sign/verify helpers, password hashing, role claims (Org Admin, Plant Manager, Maintenance Engineer, Compliance Officer, Safety Officer, Field Technician, Subject Matter Expert, External Auditor).	backend/auth
4	Onboarding flow — POST /auth/register (work-email verification before extra info) → POST /auth/login → POST /organizations (company name + industry) → POST /plants (name, city, location, type). Auto-provisions empty Knowledge Graph + storage on plant creation.	backend/api, backend/services
5	Document upload + status — POST /documents/upload (validation, metadata extraction, storage) and GET /documents/{id}/status, emitting the 'Document Uploaded' event consumed by Person 2's ingestion pipeline.	backend/services (document), backend/workers
6	Demonstration dataset loader — endpoint/script to load Person 4's pre-built sample organization (fully populated Knowledge Graph, contradictions, compliance issues, incidents) for the empty-state / judge demo path.	backend/services, docs/datasets (consumed)

7	Dashboard, Notification, Audit, Admin, Report services — GET /dashboard/alerts; org/user/role administration; audit log capture on every mutating request; report generation endpoints.	backend/services, backend/notifications
8	DevOps — Dockerfiles, docker-compose (Postgres, Redis, Neo4j, backend, frontend), GitHub Actions CI, Nginx reverse proxy config, .env.example, deploy to Railway/Render.	docker/, scripts/, docker-compose.yml

How to Test Without Others

```
# Run migrations + seed a mock org/plant/3 users
docker compose up -d postgres redis
alembic upgrade head && python scripts/seed.py
# Start API
uvicorn backend.main:app --reload
# Register -> Login -> Create Org -> Create Plant
curl -X POST localhost:8000/api/v1/auth/register -d '{...}'
curl -X POST localhost:8000/api/v1/auth/login -d '{...}'
curl -X POST localhost:8000/api/v1/organizations -H 'Authorization: Bearer <token>' -d '{...}'
curl -X POST localhost:8000/api/v1/plants -H 'Authorization: Bearer <token>' -d '{...}'
```

What You Hand to Others

To	What They Get
P2	A scoped Postgres connection + organization_id on every request, the 'Document Uploaded' event contract, and storage references for uploaded files.
P3	Every locked endpoint in the Shared Contracts section, all returning the standard response envelope, with mock-data parity from Day 1.
P4	A working demo-dataset loader endpoint so the sample organization can be triggered live during the judge demo.

P2	AI & Knowledge Engineering Lead Intelligence Layer Ingestion Pipeline Knowledge Graph GraphRAG Integrity Engines
-----------	--

You build the brain of NeuroPlant: a set of single-purpose AI services (never one giant autonomous agent) whose outputs chain into each other — Classification → OCR/Vision → Entity Extraction → Relationship Extraction → Graph Population → Knowledge Integrity → Decision Intelligence. You own the Neuro-Symbolic boundary: LLMs understand, extract, and summarize; deterministic code verifies compliance, traverses the graph, and evaluates rules. You never let an LLM decide regulatory compliance on its own.

Step	What to Build	Key Output / Module
------	---------------	---------------------

1	Document Classification Agent — categorizes every upload (SOP, Maintenance Manual, Inspection Report, Incident Report, P&ID, Engineering Drawing, Regulation, OEM Manual, Excel, Email, Work Order, Unknown) to route downstream processing.	backend/agents
2	OCR pipeline (digital PDFs skip OCR) — image enhancement, deskew, noise reduction, OCR (PaddleOCR primary, Tesseract fallback), layout/table/figure detection, reconstruction.	backend/ai (OCR Engine)
3	Vision pipeline for P&IDs — GPT-4o Vision: equipment detection, tag recognition, connection detection, equipment classification, relationship suggestions for topology (not just label reading).	backend/ai (Vision Parser)
4	Entity Extraction Agent — extracts Equipment, Procedures, Incidents, Hazards, Regulations, inspection intervals, pressures/temperatures/units, safety devices, manufacturers, dates, etc. Every entity gets UUID, type, properties, source reference, confidence, version, timestamp, organization ID.	backend/agents, backend/ai
5	Relationship Extraction Agent — discovers only the locked relationship types (see Shared Contracts) with mandatory supporting evidence; never invents a relationship.	backend/agents
6	Knowledge Graph Builder — incremental Neo4j population only (never full rebuild); each insertion records source document, page, paragraph, timestamp, confidence, version, organization.	backend/graph
7	Embedding Generator + GraphRAG Engine — text-embedding-3-large, Neo4j Vector Index; retrieval pipeline = Intent Detection → Entity Detection → Graph Traversal (queried before semantic search) → Semantic Retrieval → Evidence Collection → Decision Brief Generation.	backend/ai, backend/graph
8	Knowledge Integrity Engine — runs automatically on every graph change: Contradiction Detection (severity: Critical/Major/Minor/Informational, each with affected assets/documents/evidence/suggested	backend/compliance, backend/decision

	resolution/responsible department), Regulatory Drift Engine, Knowledge Mortality Engine (score + high-risk experts + recommended interviews), relationship completeness, orphan assets, knowledge freshness/coverage.	
9	Decision Brief Generator — every Decision Assistant answer returns a structured brief: Executive Summary, Recommendation, Operational Context, Affected Assets, Applicable Regulations, Maintenance History, Historical Incidents, Dependencies, Risk Assessment, Supporting Evidence, Confidence Level, Next Steps, Source References. Never a plain paragraph.	backend/decision
10	Expert Interview / Knowledge Capture Agent — structured interview questions (frequent failures, undocumented workarounds, difficult procedures, equipment behaving differently than docs, common new-engineer mistakes) producing Expert Insight nodes linked to equipment/procedures/incidents/maintenance.	backend/agents
11	Prompt modules — every prompt lives in shared/prompts, never inline in code; orchestrate agents with LangGraph/LangChain.	shared/prompts

How to Test Without Others

```
# Run ingestion pipeline against a local sample document
python backend/ingestion/run_pipeline.py --file sample.pdf --org-id demo-org
# Verify graph population (Neo4j Browser / cypher-shell)
MATCH (n) WHERE n.org_id = 'demo-org' RETURN n LIMIT 25;
# Query GraphRAG directly
python backend/ai/graphrag_query.py --question "Why was Pump P-204 isolated last quarter?"
# Check Knowledge Integrity output
python backend/compliance/run_integrity_scan.py --org-id demo-org
```

What You Hand to Others

To	What They Get
P1	Structured extraction results to persist as documents/audit metadata, and Knowledge Mortality / Integrity scores to surface via GET /dashboard/alerts.
P3	Decision Brief JSON for the Decision Assistant UI, graph nodes/relationships for the Graph Explorer (React Flow / Cytoscape.js), and contradiction/drift payloads for the Knowledge Integrity dashboard.

P4	The live processing visualization events (reading → extracting → graph building → integrity scoring) needed for the 'wow moment' demo, plus validated outputs against the sample dataset.
----	---

P3	Frontend Lead Experience Layer Onboarding Role-Aware Dashboards Decision Assistant UI Knowledge Graph Explorer
-----------	--

You build everything judges and users see and click. The frontend never talks to databases or AI models directly — every request goes through P1's locked REST contracts. You build 100% of the UI against mock data matching the response envelope from Day 1, then swap in live endpoints at integration.

Step	What to Build	Key Output / Module
1	Landing page — commercial-SaaS quality: problem statement, differentiators, screenshots, KPIs, 'Start Free' and 'Request Demo' CTAs.	frontend/app (marketing route)
2	Auth + onboarding flow — work-email-first signup with verification, name/password, then create-organization (name + industry) and create-first-plant (name, city, location, type) screens, conversational/minimal in tone.	frontend/app (auth, onboarding)
3	AI onboarding ('Let's teach your AI about your plant') — choice between uploading documents or loading the pre-built demo dataset; a real-time processing visualizer showing reading → entity extraction → relationship discovery → graph construction → regulation mapping → contradiction detection → integrity/mortality scoring.	frontend/features/onboarding
4	Post-processing intelligence reveal — first report shown immediately after processing: contradictions, regulatory drift, new relationships, dependencies, missing docs, orphan equipment, knowledge gaps, compliance risks, Knowledge Mortality Score. No empty dashboard.	frontend/features/dashboard
5	Role-aware dashboard routing — distinct landing dashboards per role (Org Admin, Plant Manager, Maintenance Engineer, Compliance Officer, Safety Officer, Field Technician [mobile-first], Subject Matter Expert, External Auditor [read-only]), all reading the same underlying graph.	frontend/app, frontend/features

6	Primary navigation modules — Dashboard, Knowledge Integrity, Decision Assistant, Assets, Knowledge, Documents, Compliance, Expert Knowledge, Knowledge Graph, Reports, Administration.	frontend/app (route group)
7	Decision Assistant UI — renders structured Decision Briefs (never chat bubbles): recommendation, evidence, regulations, dependencies, incidents, confidence, sources, next actions.	frontend/features/decision-assistant
8	Knowledge Graph Explorer — interactive visualization (React Flow preferred, Cytoscape.js alternative) for equipment relationships, dependencies, document lineage, regulation mapping.	frontend/features/graph-explorer
9	Compliance, Expert Knowledge, Reports, Administration screens — regulatory coverage/audit views, guided expert interview UI, executive/compliance/integrity/mortality report views, user/role/plant/API-key admin screens.	frontend/features/*

How to Test Without Others

```
# Frontend dev server against mocked API responses matching the envelope
npm run dev
# Point fetch/Axios base URL at a local mock server (msw or json-server) returning:
# { success, data, message, timestamp } for every locked endpoint
# Verify each role-aware dashboard renders from mock org/plant/document fixtures
```

What You Hand to Others

To	What They Get
P1 / P2	UI-driven request shapes for every locked endpoint (pagination, filters) so backend responses are never guessed at integration time.
P4	Screens ready for the demo script: landing → onboarding → live processing → intelligence reveal → dashboard → contradiction → regulatory drift → Decision Brief → Graph Explorer.

P4	Architecture & Product Lead Chief Architect Documentation Sample Dataset Presentation Demo Strategy PR Review
-----------	---

You protect the product vision. You don't redesign the platform unless a fundamental flaw is demonstrated, and you make sure every feature traceably maps to the judging rubric

(Innovation, Business Impact, Technical Excellence, Scalability, User Experience). You author the demo dataset that solves the empty-state problem for both judges and first-time users.

Step	What to Build	Key Output / Module
1	Architecture diagrams — the six logical layers (Experience, Application, Intelligence, Knowledge, Data, Infrastructure), the document ingestion pipeline, the GraphRAG retrieval pipeline, and the event-driven processing diagram (Document Uploaded → Graph Update → Relationship Recalculation → Regulatory Drift → Knowledge Integrity → Dashboard Refresh → Notifications).	docs/architecture, docs/diagrams
2	ER diagram + Neo4j schema documentation — Postgres core tables and the locked Neo4j node/relationship lists from Shared Contracts, kept as the single source of truth P1 and P2 implement against.	docs/architecture
3	Demonstration / sample dataset — a fully populated demo organization: realistic industrial documents, populated Knowledge Graph, pre-seeded contradictions, compliance issues, historical incidents, expert knowledge, work orders, inspections, regulations.	docs/datasets
4	API documentation — OpenAPI/Swagger pass-through review, ensuring every endpoint matches the locked list and response envelope exactly.	docs/api
5	Presentation deck + architecture-diagram-as-pitch — mapped explicitly to the judging rubric: Innovation (Knowledge Integrity Engine, Mortality Score, Regulatory Drift, Decision Briefs), Business Impact (fragmentation, knowledge loss, compliance, decision latency), Technical Excellence (GraphRAG, Neo4j, Temporal KG, Neuro-Symbolic AI, multi-modal ingestion), Scalability (cloud-native, multi-tenant, modular), UX (role-aware dashboards, modern onboarding, Decision Brief UX).	Presentation Deck deliverable
6	Demo script — landing page → create organization → upload/load sample plant → live AI processing → first operational insights → dashboard → Knowledge Integrity Score → Knowledge Mortality Score → open a contradiction → open Regulatory Drift → ask an operational question → generate a Decision	Demo Video script

	Brief → explore the Knowledge Graph. Intelligence before conversation, always.	
7	PR / architectural review — every merge reviewed for adherence to Core Philosophy, Architecture Rules, and Shared Contracts before it lands on develop.	Cross-cutting

How to Test Without Others

```
# Validate the sample dataset independently of any running backend
python scripts/validate_dataset.py docs/datasets/demo_org.json
# Render architecture diagrams from source (e.g. Mermaid/diagrams-as-code) for the deck
# Dry-run the demo script against staged screenshots before any code is wired
```

What You Hand to Others

To	What They Get
P1	The demo dataset payload shape to wire into the dataset-loader endpoint, and the API documentation to validate against the locked contracts.
P2	The exact sample documents and expected contradictions/drift/mortality outcomes to validate extraction and integrity-engine accuracy.
P3	Screenshots, KPI copy, and the full demo script sequence to build the landing page and guide which screens need polish first.

Integration Plan

Integration Step	Who Merges	What Gets Tested	Definition of Done
Step 1: P1 + P2	P2 pulls P1's branch, runs migrations and connects ingestion to live Postgres/Neo4j	Upload a document → pipeline runs → entities/relationships land in Neo4j scoped to organization_id	A real upload produces a populated graph node set with correct org scoping and source references
Step 2: P1+P2 + P3	P3 pulls merged branch, replaces mock fetches with the live locked endpoints	Onboarding flow end-to-end; dashboard alerts, contradictions, drift, and mortality score render from live data	Full onboarding → processing visualization → intelligence reveal works without mock data
Step 3: All + P4	P4 loads the demo dataset into the merged build and rehearses the full demo script	Landing → org/plant creation → demo dataset load → dashboard → contradiction → regulatory drift → Decision Brief → Graph Explorer	Every role-aware dashboard works; Decision Briefs are structured (not paragraphs); demo is judge-ready

Final Summary

Member	Produces	Reads	Test Independently By
P1	Locked auth/org/plant/document/dashboard endpoints, Postgres schema, audit/notification services	Nothing live — starts fresh	Seeded local Postgres + mock-envelope responses
P2	Knowledge Graph population, GraphRAG answers, Decision Briefs, Integrity/Drift/Mortality outputs	P1's organization_id-scoped storage + document events (use sample dataset)	docs/datasets sample organization, run locally without live Postgres
P3	Full role-aware UI: onboarding, dashboards, Decision Assistant, Graph Explorer, Reports, Admin	P1's locked endpoints + P2's Decision Brief / Graph shapes (use mocks)	Mock JSON matching the standard response envelope
P4	Demo dataset, architecture docs, diagrams, presentation deck, demo script	Progress/state from all three other members	Validating the dataset and diagrams independent of running code



Integration happens in 3 phases, late in the hackathon. If the Shared Contracts (Postgres schema, Neo4j labels/relationships, API envelope, locked endpoints) were respected, it works the first time. Ship it.

Stitch Prompts — Individual AI Context per Member

Each member pastes ONLY their own prompt below into their AI coding assistant. Each prompt is self-contained, scoped strictly to that member's files, and encodes the full Core Philosophy, Architecture Rules, and Shared Contracts so no member needs to re-explain project context.

P1 — Stitch Prompt — Backend Lead (Application Layer, Auth, DevOps)

You are building ONLY the FastAPI application layer for NeuroPlant, an Industrial Knowledge Intelligence Platform (NOT a chatbot, NOT a PDF Q&A app, NOT a document repository).

Your scope is STRICTLY limited to: backend/api, backend/auth, backend/core, backend/database, backend/models, backend/schemas, backend/services (org, plant, user, document, notification, admin, audit, report — business logic, not raw AI logic), backend/workers, backend/notifications, docker/, scripts/, docker-compose.yml.

DO NOT create or modify: backend/agents, backend/graph, backend/ai, backend/ingestion, backend/compliance, backend/decision (Person 2 owns the Intelligence Layer), or anything under frontend/.

Stack: FastAPI, Python 3.12+, Uvicorn, Pydantic, JWT auth, RBAC, Celery or FastAPI BackgroundTasks, OpenAPI/Swagger, FastAPI Depends for DI. Databases: PostgreSQL (primary), Redis (cache/sessions/jobs). Object storage for raw documents (filesystem locally).

Mandatory PostgreSQL tables (minimum): organizations, plants, users, roles, documents, audit_logs — every table needs organization_id and created_at/updated_at.

Every API response uses this exact envelope, no exceptions:

```
{ "success": true, "data": {}, "message": "string", "timestamp": "ISO8601" }
```

Implement these locked endpoints exactly (Person 3 builds against them from Day 1 with mocks):

POST /api/v1/auth/register, POST /api/v1/auth/login, POST /api/v1/organizations, POST /api/v1/plants, POST /api/v1/documents/upload, GET /api/v1/documents/{id}/status, GET /api/v1/dashboard/alerts.

(integrity/mortality/decision/graph/expert endpoints are implemented by Person 2's services but exposed through your api/ routing layer using the same envelope.)

Multi-tenancy is mandatory: every query is scoped by the authenticated user's organization_id. RBAC roles: Organization Administrator, Plant Manager, Maintenance Engineer, Compliance Officer, Safety Officer, Field Technician, Subject Matter Expert, External Auditor.

Onboarding sequence to implement: work-email-first registration with email verification BEFORE collecting name/password -> create organization (name + industry) -> create first plant (name, city, location, type) -> auto-provision an empty Knowledge Graph + storage + assign first user as Org Admin.

On document upload, emit a 'Document Uploaded' event that Person 2's ingestion pipeline consumes — do not perform any AI inference yourself.

Coding standards: snake_case Python, REST APIs, no business logic in route handlers (delegate to services), no AI prompts anywhere in this layer, never hardcode secrets — use environment variables, validate all incoming/outgoing data with Pydantic.

Git workflow: main -> develop -> feature/* (e.g. feature/auth, feature/document-upload). No direct commits to main.

Test independently: docker compose up postgres redis; run migrations + seed script; curl register/login/organizations/plants against the envelope shape above.

What you hand to others: Person 2 gets a scoped Postgres connection, organization_id on every request, and the document-uploaded event contract. Person 3 gets every locked endpoint above returning the standard envelope with mock-data parity from Day 1.

P2 — Stitch Prompt — AI & Knowledge Engineering Lead

You are building ONLY the Intelligence Layer for NeuroPlant. This is explicitly NOT a chatbot and NOT a simple RAG application — it is an intelligence-first platform: Documents -> AI Understanding -> Entity Extraction -> Relationship Discovery -> Temporal Knowledge Graph -> Knowledge Integrity Engine -> Operational Intelligence -> Decision Brief -> User.

Your scope is STRICTLY limited to: backend/agents, backend/graph, backend/ai, backend/ingestion, backend/compliance, backend/decision, shared/prompts.

DO NOT create or modify: backend/api, backend/auth, backend/core, backend/database, backend/services (Person 1 owns those), or anything under frontend/.

AI stack: Primary LLM GPT-4.1 or GPT-4o (future-compatible with Claude, Gemini, Llama, DeepSeek). Embeddings: text-embedding-3-large. OCR: PaddleOCR primary, Tesseract fallback. Vision: GPT-4o Vision. Document parsing: LlamaParse / Unstructured.io. Orchestration: LangChain + LangGraph. Knowledge store: Neo4j (mandatory) with Neo4j Vector Index for embeddings.

Build single-purpose services that chain — never one giant autonomous agent doing everything: Document Classification Agent, OCR Agent, Vision Parsing Agent, Entity Extraction Agent, Relationship Extraction Agent, Knowledge Graph Builder, GraphRAG Retrieval Agent, Decision Brief Generator, Contradiction Detection Engine, Regulatory Drift Engine, Knowledge Mortality Engine, Expert Knowledge Capture Agent, Knowledge Integrity Engine.

Document ingestion pipeline (every document, no bypassing): Upload -> Validation -> Metadata Extraction -> Storage -> OCR (skip for digital PDFs) -> Document Classification -> Chunking -> Entity Extraction -> Relationship Extraction -> Embedding Generation -> Graph Population -> Knowledge Integrity Scan -> Decision Intelligence Update -> Dashboard Refresh.

Document classification categories: SOP, Maintenance Manual, Inspection Report, Incident Report, P&ID, Engineering Drawing, Regulation, OEM Manual, Excel, Email, Work Order, Unknown.

P&ID/vision pipeline must understand topology, not just read labels: Image -> Vision Model -> Equipment Detection -> Tag Recognition -> Connection Detection -> Equipment Classification -> Relationship Suggestions -> Graph Population.

Every extracted entity gets: UUID, Entity Type, Properties, Source Reference, Confidence, Version, Timestamp, Organization ID.

Locked Neo4j node labels (do not invent new ones): Equipment, Procedure, Regulation, Inspection, Incident, WorkOrder, MaintenanceHistory, Expert, ExpertInsight, Document, Plant, Organization, PIDElement. Every node needs org_id, created_at, updated_at, confidence, source_document_id.

Locked Neo4j relationship types (do not invent new ones, every relationship needs supporting evidence): CONNECTED_TO, PART_OF, LOCATED_IN, MAINTAINED_BY, GOVERNED_BY, REFERENCES, INSPECTED_BY, CAUSES, PREVENTS, DEPENDS_ON, DOWNSTREAM_AFFECTS, REQUIRES, AUTHORED_BY, CONTRADICTS, SUPERSEDES, RELATES_TO.

Graph updates are always incremental — never rebuild the whole graph on a new upload. Each insertion records creation source, document, page, paragraph, timestamp, confidence, version, organization.

GraphRAG retrieval pipeline (graph is queried before semantic search whenever possible): User Question -> Intent Detection -> Entity Detection -> Graph Traversal -> Semantic Retrieval -> Evidence Collection -> Decision Brief Generation -> LLM Formatting -> Response.

Neuro-Symbolic boundary (strict): LLMs do understanding, extraction, classification, relationship suggestion, summarization, natural language generation. Deterministic code does compliance verification, graph traversal, rule evaluation, business logic, permission checks, temporal calculations. Never let an LLM determine regulatory compliance by itself.

Knowledge Integrity Engine runs automatically on every graph change and covers: Contradiction Detection (classify Critical/Major/Minor/Informational; each contradiction needs description, affected assets, affected documents, evidence, severity, suggested resolution, responsible department), Regulatory Drift Detection (compare new/updated regulations against linked SOPs/procedures/inspection schedules; output outdated SOPs, changed

intervals, new mandatory requirements, superseded clauses, unmapped regulations, compliance impact summary), Knowledge Mortality Engine (score quantifying how much knowledge depends on few experts; output score, high-risk experts, knowledge at risk, affected assets, recommended interviews), relationship completeness, orphan assets, knowledge freshness/coverage.

Decision Brief Generator: every answer is a structured object, never a paragraph: Executive Summary, Recommendation, Operational Context, Affected Assets, Applicable Regulations, Maintenance History, Historical Incidents, Dependencies, Risk Assessment, Supporting Evidence, Confidence Level, Suggested Next Steps, Source References.

Expert Knowledge Capture Agent: guided interview agent asking senior engineers structured questions (frequent failures, undocumented workarounds, difficult procedures, equipment behaving differently than docs, common new-engineer mistakes), converting answers into Expert Insight nodes linked to equipment/procedures/incidents/maintenance.

Event-driven processing: Document Uploaded, Graph Updated, New Regulation Added, New Incident Added, Inspection Completed, Procedure Updated, Expert Interview Completed — each triggers downstream recalculation (e.g. Procedure Updated -> Graph Update -> Relationship Recalculation -> Regulatory Drift -> Knowledge Integrity -> Dashboard Refresh -> Notifications).

Coding standards: snake_case Python, prompts live ONLY in shared/prompts (never inline in agent code), every AI output validated before being persisted to Neo4j, single responsibility per service, dependency injection where appropriate.

Test independently: run the ingestion pipeline against a sample document and the docs/datasets demo organization without needing a live Postgres; inspect Neo4j directly via Cypher; query GraphRAG and the integrity scan as standalone scripts.

What you hand to others: Person 1 gets structured extraction results and Integrity/Mortality scores to surface on dashboards. Person 3 gets Decision Brief JSON, graph nodes/relationships for the Graph Explorer, and contradiction/drift payloads. Person 4 gets live processing-stage events for the onboarding 'wow moment' visualization and validated outputs against the sample dataset.

P3 — Stitch Prompt — Frontend Lead

You are building ONLY the frontend Experience Layer for NeuroPlant — an Industrial Knowledge Intelligence Platform that is explicitly NOT a chatbot. Conversation (the Decision Assistant) is one interaction mechanism among many; the real product is the continuously evolving Knowledge Graph and intelligence surfaced through role-aware dashboards.

Your scope is STRICTLY limited to: frontend/app, frontend/components, frontend/features, frontend/hooks, frontend/lib, frontend/services, frontend/types, frontend/utils, frontend/public.

DO NOT create or modify: any backend/ folder, shared/prompts, or any database schema. Never call Neo4j/Postgres/AI models directly — every request goes through the backend REST API using the standard envelope.

Stack: Next.js 15, TypeScript, React 19, TailwindCSS, shadcn/ui, Lucide React icons, Recharts, React Flow (preferred) or Cytoscape.js for the Graph Explorer, Zustand for client state, TanStack Query for server state, React Hook Form + Zod for forms/validation, Framer Motion for animation, Axios for HTTP, JWT delivered via HTTP-only cookies.

Every API response you consume has this envelope — code against it exactly:

```
{ "success": true, "data": {}, "message": "string", "timestamp": "ISO8601" }
```

Build against these locked endpoints with mocks on Day 1, then swap to live calls at integration: POST /api/v1/auth/register, POST /api/v1/auth/login, POST /api/v1/organizations, POST /api/v1/plants, POST /api/v1/documents/upload, GET /api/v1/documents/{id}/status, GET /api/v1/dashboard/alerts, GET /api/v1/integrity/contradictions, GET /api/v1/integrity/regulatory-drift, GET /api/v1/mortality/score, POST /api/v1/decisions/query, GET /api/v1/graph/nodes, GET /api/v1/graph/relationships, POST /api/v1/expert/interview/start.

Landing page: must look like a commercial SaaS product (not a university project) — explain the problem, key differentiators, screenshots, major KPIs, with 'Start Free' and 'Request Demo' CTAs. Modern, clean, industrial, enterprise visual language.

Onboarding flow, in exact order: work email only -> email verification -> full name + password -> create organization (company name + industry: Oil & Gas, Manufacturing, Steel, Mining, Power Generation, Chemical Processing, Construction, Utilities, Other) -> create first plant (name, city, location, plant type) -> AI onboarding screen 'Let's teach your AI about your plant' offering two choices: upload your own documents, or load the pre-built demonstration dataset.

Live processing visualization (a deliberate 'wow moment', not a spinner): show the system reading documents, extracting entities, discovering relationships, constructing the Knowledge Graph, mapping regulations, identifying contradictions, computing Knowledge Integrity metrics, calculating the Knowledge Mortality Score, generating operational intelligence — as it actually happens.

Immediately after processing, show the first intelligence report (not an empty dashboard): discovered contradictions, regulatory drift, new relationships, dependencies, missing documentation, orphan equipment, knowledge gaps, compliance risks, current Knowledge Mortality Score.

Role-aware dashboard routing — each role lands on a different home screen: Organization Administrator -> organization overview, Plant Manager -> operational health dashboard, Maintenance Engineer -> maintenance workspace, Compliance Officer -> compliance dashboard, Safety Officer -> safety dashboard, Field Technician -> simplified mobile-first

interface, Subject Matter Expert -> Expert Knowledge workspace, External Auditor -> read-only audit dashboard. All roles read the same underlying graph.

Primary navigation (intelligence-first, not search-first — answers 'what requires my attention today', not 'what would you like to search for'): Dashboard, Knowledge Integrity, Decision Assistant, Assets, Knowledge, Documents, Compliance, Expert Knowledge, Knowledge Graph, Reports, Administration.

Dashboard: Knowledge Integrity Score, Knowledge Mortality Score, critical alerts, regulatory drift notifications, recent AI discoveries, operational KPIs, unresolved contradictions, cross-functional insights.

Decision Assistant: NOT chat bubbles. Renders a structured Decision Brief card for every query — Executive Summary, Recommendation, Operational Context, Affected Assets, Applicable Regulations, Maintenance History, Historical Incidents, Dependencies, Risk Assessment, Supporting Evidence, Confidence Level, Suggested Next Steps, Source References.

Assets: full digital profile per equipment — specs, maintenance history, inspections, incidents, procedures, regulations, drawings, connected documents, downstream dependencies, expert insights, graph relationships.

Knowledge module: browse procedures/regulations/inspections/incidents/standards/work orders/maintenance strategies/expert knowledge by relationship, not by folder.

Documents: supporting workspace only — upload, processing status, extraction metadata/quality. Documents are evidence, never the primary product — don't make this the main navigation focus.

Compliance: regulatory drift, missing evidence, outdated procedures, inspection coverage, regulatory mappings, audit reports.

Expert Knowledge: structured AI-guided interview UI capturing tribal knowledge before retirement, linked to assets/procedures/incidents/maintenance.

Knowledge Graph Explorer: interactive React Flow / Cytoscape.js visualization of equipment relationships, dependencies, document lineage, regulation mapping — for advanced/technical users.

Reports: executive summaries, compliance reports, Knowledge Integrity reports, Knowledge Mortality reports, contradiction analysis, operational intelligence summaries, audit packages.

Administration: users, permissions, plants, integrations, AI settings, storage, API keys, audit logs, org configuration (Org Admin only).

Coding standards: camelCase TypeScript, PascalCase React components, Zod-validated forms, no business logic duplicated from backend, no direct DB/AI calls.

Test independently: run against a local mock server (msw / json-server) returning the exact envelope shape for every locked endpoint listed above; verify each role lands on its correct dashboard and the onboarding 'wow moment' visualization renders with fixture data.

What you hand to others: Person 1/2 get the exact request shapes (filters, pagination) your screens need so responses are never guessed at integration. Person 4 gets demo-ready screens for the full presentation script.

P4 — Stitch Prompt — Architecture & Product Lead

You are acting as Lead Enterprise Architect, Principal AI Engineer reviewer, Product Architect, and Technical Reviewer for NeuroPlant, an AI-Powered Industrial Knowledge Intelligence Platform (ET AI Hackathon 2026, Problem Statement 8). Your job is not to write feature code — it is to preserve architectural consistency and produce the artifacts that make the product legible to judges.

Your scope is STRICTLY limited to: docs/architecture, docs/api (review only), docs/diagrams, docs/datasets, the Presentation Deck, the Demo Video script, and pull-request architectural review across all other members' work.

DO NOT write backend/frontend feature code yourself. DO NOT redesign previously approved architecture unless a genuine critical flaw is demonstrated — improvements must be evolutionary, not revolutionary. Never let the platform get simplified into 'just another RAG chatbot.'

Core philosophy you enforce on every review: this is NOT a chatbot, NOT a PDF Q&A system, NOT a document repository, NOT a simple RAG application, NOT a replacement for ERP/CMMS/EDMS/SharePoint/SAP. It IS an Industrial Knowledge Intelligence Platform transforming fragmented industrial knowledge into explainable operational intelligence using AI, Knowledge Graphs, GraphRAG, and deterministic reasoning. Documents are evidence; Knowledge is the product; Operational Intelligence is the output.

Six architecture layers to document and diagram: Experience (Next.js/React frontend), Application (FastAPI orchestration, no direct AI inference), Intelligence (single-purpose AI agents chained together — never one giant agent), Knowledge (Neo4j temporal graph, mandatory), Data (PostgreSQL, Redis, object storage, vector index), Infrastructure (Docker, Docker Compose, GitHub Actions, Railway/Render, Nginx).

Document the immutable Shared Contracts as the single source of truth for the other three members: PostgreSQL core tables (organizations, plants, users, roles, documents, audit_logs — each with organization_id, created_at, updated_at); locked Neo4j node labels (Equipment, Procedure, Regulation, Inspection, Incident, WorkOrder, MaintenanceHistory, Expert, ExpertInsight, Document, Plant, Organization, PIDElement — each with org_id, created_at,

updated_at, confidence, source_document_id); locked Neo4j relationship types (CONNECTED_TO, PART_OF, LOCATED_IN, MAINTAINED_BY, GOVERNED_BY, REFERENCES, INSPECTED_BY, CAUSES, PREVENTS, DEPENDS_ON, DOWNSTREAM_AFFECTS, REQUIRES, AUTHORED_BY, CONTRADICTS, SUPERSEDES, RELATES_TO); the API response envelope; and the locked endpoint list (auth/register, auth/login, organizations, plants, documents/upload, documents/{id}/status, dashboard/alerts, integrity/contradictions, integrity/regulatory-drift, mortality/score, decisions/query, graph/nodes, graph/relationships, expert/interview/start).

Build the demonstration/sample dataset: a fully populated demo organization with realistic industrial documents, a populated Knowledge Graph, pre-seeded contradictions, compliance issues, historical incidents, expert knowledge, work orders, inspections, and regulations — this solves the empty-state problem for judges and first-time users and must be loadable instantly without waiting on live AI processing.

Map every feature explicitly to the judging rubric in your deck: Innovation (Knowledge Integrity Engine, Knowledge Mortality Score, Regulatory Drift Detection, Decision Briefs); Business Impact (knowledge fragmentation, knowledge loss, compliance, operational decision latency); Technical Excellence (GraphRAG, Neo4j, Temporal Knowledge Graph, Neuro-Symbolic AI, multi-modal ingestion); Scalability (cloud-native SaaS, multi-tenancy, modular architecture, independent AI services); User Experience (role-aware dashboards, modern onboarding, operational dashboards, Decision Brief UX, minimal friction).

Write the demo script in this exact order — intelligence before conversation, always: landing page -> create new organization -> upload sample documents OR load the sample plant -> show live AI processing -> reveal first operational insights -> open dashboard -> show Knowledge Integrity Score -> show Knowledge Mortality Score -> open a contradiction -> open Regulatory Drift -> ask an operational question -> generate a Decision Brief -> finally explore the Knowledge Graph.

PR review checklist you apply to every other member's merge: preserves architectural consistency; doesn't reintroduce a chatbot framing; doesn't remove Neo4j/GraphRAG; doesn't replace deterministic verification with LLM reasoning; doesn't break multi-tenancy or RBAC; doesn't invent graph relationships without evidence; stays realistic for the 10-day timeline; prompts live only in shared/prompts, never inline in controllers; every API response uses the standard envelope.

What you hand to others: Person 1 gets the dataset payload shape for the loader endpoint and the reviewed API docs. Person 2 gets the exact sample documents plus expected contradiction/drift/mortality outcomes to validate extraction accuracy. Person 3 gets screenshots, KPI copy, and the full demo sequence to prioritize screen polish.